# Relatório Técnico: Teoria dos Conjuntos na Prática Computacional

**Disciplina:** Matemática Concreta | **Curso:** Desenvolvimento de Software Multiplataforma – Fatec Osasco
**Tema:** Laboratório de Conjuntos e Conjuntos Numéricos

---

## 1. Onde a Teoria dos Conjuntos apareceu no código?

A Teoria dos Conjuntos não atuou como um conceito abstrato distante, mas sim como a **estrutura central de tomada de decisão, validação de regras de negócio e modelagem de dados** ao longo de todo o projeto:

- **Eliminação de Redundâncias e Unicidade:** Na computação básica, listas aceitam duplicatas (ex.: `[1, 2, 2, 3]`). A própria definição de conjunto (onde a multiplicidade de repetição de elementos não altera a coleção) exigiu a criação de um algoritmo de desduplicação (`remover_duplicatas`), garantindo que apenas dados únicos sejam processados, refletindo o rigor de conjuntos matemáticos.
- **Operações Fundamentais como Mecanismo Lógico:** A **União (A ∪ B)** foi usada para agregação de dados e acúmulo de privilégios; a **Interseção (A ∩ B)** funcionou como filtro de similaridade e compatibilidade entre interesses; e a **Diferença (A − B)** e o **Complemento (U − A)** atuaram como detecção de pendências e busca por elementos faltantes.
- **Hierarquia e Classificação Numérica (Inclusão):** Na Parte 2, a cadeia de inclusão matemática N ⊂ Z ⊂ Q ⊂ R e I ⊂ R serviu como motor de regras de inferência lógica. Provar que um número pertence a N implica imediatamente que ele pertence a Z, Q e R.
- **Aplicações em Engenharia de Software (Parte 3):** No *Índice de Similaridade de Jaccard*, a razão entre a cardinalidade da interseção e da união (|A ∩ B| / |A ∪ B|) permitiu transformar interesses de perfis em um coeficiente objetivo de afinidade. No *Controle de Acesso RBAC*, permissões foram modeladas como conjuntos: múltiplos papéis equivalem à **União de Perfis**, hierarquia de cargos equivale a **Subconjuntos (editor ⊆ admin)** e autorização equivale à **Pertinência (ação ∈ Permissões)**.

## 2. Qual operação foi mais difícil de implementar e por quê?

A operação mais desafiadora do projeto foi a **Identificação de Dízimas Periódicas com Dedução Algorítmica da Fração Geratriz**, acompanhada do rigor de **implementação purista das operações sem operadores nativos**:

- **O Desafio Algébrico da Fração Geratriz:** Enquanto frações comuns (a/b) ou decimais exatos dependem de operações aritméticas diretas, a dízima periódica (em especial a composta, como `1,2(5)`) exige a separação estrutural de três partes: a parte inteira (I), o anti-período não repetente (N, com k dígitos) e o período repetente (P, com p dígitos). Foi necessário programar a automação do método do cancelamento algébrico — multiplicando por $10^{k+p}$ e por $10^{k}$ e subtraindo as equações para eliminar a cauda infinita —, seguido da simplificação pelo Algoritmo de Euclides (MDC) para obter a fração irredutível 113/90. Tratar o caso curioso de `0,(9)` (cuja geratriz matemática resulta em 9/9 = 1, promovendo o número de decimal para o conjunto dos Naturais) exigiu modelagem matemática precisa.
- **A Restrição do Algoritmo Manual:** A exigência de não usar operadores prontos do Python (`|`, `&`, `-`, `^`, `issubset`) exigiu reconstruir tudo com laços `for` e condicionais `if`. A verificação de **Subconjunto Próprio (A ⊂ B)** exigiu checar que todo elemento de A pertence a B e, simultaneamente, que a cardinalidade de A é estritamente menor que a de B, tratando rigorosamente casos de borda como o conjunto vazio (onde ∅ ⊆ B é sempre verdadeiro, mas ∅ ⊂ ∅ é falso).

> **Conclusão:** A reconstrução manual das operações consolidou na prática a relação direta entre a Teoria dos Conjuntos e a ciência da computação: desde a prevenção de bugs de borda até o projeto de algoritmos eficientes de busca, sistemas de recomendação e segurança de sistemas.